

INSTITUTE OF SPACE TECHNOLOGY
KICSIT, Kahuta Campus



WEB TECHNOLOGY LAB

Assignment no .01

Name	Ayesha Rahman
Reg No	242201006
Class	CS-5A
Professor	Sir Hussain Gillani

Department of Computer Science

Smart Campus Student Portal
Project Documentation

1. Problem Statement
Students currently rely on scattered sources to find course information, faculty details, announcements, academic activities, class schedules, and assignment deadlines. This fragmentation causes confusion, missed deadlines, and wasted time.

The university requires a single, modern, user-friendly web portal that brings all essential academic information into one place — accessible from any device.

2. Proposed Solution
A Smart Campus Student Portal — a responsive, interactive front-end web application built with HTML5, CSS3, and JavaScript. It consolidates academic data into a clean interface with dynamic content, search/filter capabilities, form validation, task management, and persistent browser storage.

3. Objectives

- Combine semantic HTML, modern CSS, and JavaScript into one functional application
- Demonstrate responsive design across desktop, laptop, tablet, and mobile
- Implement dynamic content, DOM manipulation, and event handling
- Provide searching, filtering, and form validation
- Use localStorage for basic client-side persistence
- Convert a real-world client problem into a usable web solution

4. Functional Requirements

#	Feature	Description
1	Home Page	University branding, responsive nav, hero with CTA, welcome section, announcements, quick-access cards, upcoming events, professional footer
2	Student Dashboard	Cards for Registered Courses, Completed Courses, Current Semester, Attendance %, Upcoming Assignments, Upcoming Events — JS-generated
3	Course Management	JS array of 6+ course objects (Code, Name, Instructor, Credit Hours, Semester, Status)
4	Course Search & Filter	Search by name/code; filter by semester or status; updates without page refresh
5	Weekly Class Schedule	Timetable with Course, Instructor, Day, Start/End Time, Classroom/Lab; responsive table/card layout

6	Assignment / Task Manager	Add tasks (Title, Course, Due Date, Priority); mark complete; delete; visual distinction; feedback messages
7	Browser Local Storage	Tasks persist after refresh; theme preference also saved
8	Student Registration / Profile Form	Full Name, Reg No., Email, Department, Semester, Phone, Password, Confirm Password — with inline JS validation
9	Announcements	JS array (Title, Description, Date, Category); dynamic rendering; category filtering
10	Light / Dark Mode	JS + CSS class toggle; preference saved in localStorage
11	Responsive Design	Flexbox, CSS Grid, media queries, responsive units; no horizontal scroll

5. JavaScript Concepts Demonstrated

- let / const variables
- Data types and operators
- Conditional statements and loops
- Functions (reusable, named)
- Arrays and objects
- Array methods — map, filter, find, forEach
- DOM selection and manipulation
- Event listeners
- Dynamic HTML generation
- Form validation (inline messages, not just alerts)
- Searching and filtering
- localStorage

6. Project Structure

```
smart-campus-portal/
├── index.html
├── courses.html
├── schedule.html
├── tasks.html
├── profile.html
├── dashboard.html
├── document.docx
│
├── images/
│   └── kicsitLogo.svg
├── css/
```

```
│   └── style.css
│
└── js/
    ├── data.js
    ├── main.js
    ├── profile.js
    ├── scedule.js
    ├── theme-init.js
    ├── courses.js
    └── tasks.js
```

7. User Experience Requirements

- Clear feedback messages — e.g., "Task added successfully", "No matching course found"
- Consistent typography, spacing, navigation, buttons, and forms
- Clear visual hierarchy across all pages
- Responsive on desktop, laptop, tablet, and mobile

8. Professional Development Standards

- Clean, consistent user interface
- Semantic, well-structured HTML
- Reusable CSS (no unnecessary duplication)
- Meaningful variable and function names
- Proper indentation and readable code
- Useful comments where necessary
- Logical file/folder organization
- Functional JavaScript interactions
- Independent design and implementation

9. Problem → Solution Mapping

Problem	Requirement	Interface	JS Logic	Result
Scattered info	One unified portal	Multi-page site	DOM rendering	Centralized access
Many tasks to track	Task manager	Input form + list	Add / complete / delete	Tasks persist after refresh
Hard to find courses	Search & filter	Search bar + filters	filter(), event listeners	Instant results
Weak form input	Validated registration	Form with inline errors	Regex + conditionals	Clean data entry

Harsh UI in dark rooms	Theme toggle	Button/switch	Class toggle	Remembered preference

10. How to Run

Download or clone the project folder
Open index.html in any modern browser (Chrome, Firefox, Edge, Safari)
No build step, no dependencies — pure HTML, CSS, and JavaScript
All navigation links and interactive features have been tested before delivery.